

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
I	INTRODUCTION	3
	OBJECTIVES	4
	SCOPE OF THE PROJECT	5
	EXISTING SYSTEM	6
	PROPOSED SYSTEM	7
		9
II	PROJECT DESCRIPTION	9
	METHODOLOGY	10
	TECHNIQUES ✓	12
III	SYSTEM REQUIREMENTS	14
	FUNCTIONAL & NON-FUNCTIONAL REQUIREMENT	14
	SOFTWARE ✓	14
	HARDWARE	15
IV	MODULES ✓	17
V	SYSTEM DESIGN	19
	DATA FLOW DIAGRAM	20
VI	SOFTWARE SPECIFICATION	22
VII	SAMPLE SOURCE CODE	29
VIII	SCREENSHOTS	35
IX	TESTING	39
X	FEASIBILITY STUDY	45
XI	FUTURE ENHANCEMENT ✓	51
XII	CONCLUSION	57
XIII	REFERENCES	59

DURSUYRE
MUSIC STREAMING WEB APPLICATION

ABSTRACT

Dubsvibe is a web-based music streaming application designed to provide users with an easy and convenient platform to listen to music online. The main objective of this project is to develop a simple, efficient, and user-friendly music streaming system where users can browse, play, and manage songs through a web interface. The application focuses on delivering smooth music playback while maintaining a clean and responsive user experience. The frontend of the Dubsvibe application is developed using HTML, CSS, and JavaScript, which helps in creating an interactive and visually appealing interface. HTML is used to structure the web pages, CSS is responsible for designing and styling the layout, and JavaScript is used to implement dynamic features such as music controls, playlist handling, and user interactions. These technologies together ensure that the application works smoothly across different web browsers. The backend of the system is built using Python with the Flask framework. Flask is a lightweight and flexible web framework that enables efficient development of web applications. In this project, Flask handles server-side operations such as processing user requests, managing music files, handling playlists, and connecting the application with the database. The backend ensures proper communication between the frontend interface and the stored data. For data storage, the system uses SQLite, a lightweight relational database management system. SQLite is used to store important information such as song details, artist names, album information, and user playlists. Since SQLite does not require a separate server, it is well suited for small to medium web applications like Dubsvibe. The database helps in organizing and retrieving music data efficiently whenever the user searches or plays a song.

INTRODUCTION

Music streaming has become one of the most popular ways for people to listen to music in the digital era. With the rapid growth of the internet and web technologies, users prefer online platforms where they can easily access and play their favorite songs anytime and anywhere. Traditional methods of storing music on personal devices are gradually being replaced by webbased music streaming applications that offer convenience, accessibility, and better user experience.

The Dobsvibe Music Streaming Web Application is developed to provide a simple and efficient platform where users can stream music through a web browser. The main aim of this project is to design and implement a web application that allows users to browse available songs, play music instantly, and manage playlists through an interactive interface. The system focuses on delivering smooth audio playback while maintaining a user-friendly design.

The application is developed using modern web technologies. The frontend is built using HTML, CSS, and JavaScript, which are used to design the layout, style the interface, and add dynamic functionalities. These technologies help in creating a responsive and interactive user interface that improves the overall user experience.

The backend of the system is developed using Python with the Flask framework, which handles the server-side logic of the application. Flask is a lightweight web framework that simplifies web development and enables smooth communication between the frontend and the database. It processes user requests, manages song data, and ensures proper functioning of the application.

For storing and managing music-related information, the system uses SQLite as the database. SQLite is a lightweight relational database that efficiently stores song details, artist information, and other necessary data. It allows the system to retrieve music information quickly whenever the user searches or plays a song. This project helps in understanding fullstack web development and shows how different technologies can work together to create a functional music streaming platform. The system can also be further enhanced in the future by adding features such as user login, personalized playlists, music recommendations, and cloudbased music storage.

OBJECTIVES

The main objective of the Dubsvibe Music Streaming Web Application is to develop a simple and efficient web-based platform that allows users to stream and manage music easily through a web browser. The system is designed to provide a smooth and user-friendly music listening experience by integrating frontend, backend, and database technologies.

The key objectives of this project are:

- To develop a web-based music streaming platform that allows users to listen to songs online without downloading them.
- To create an interactive and responsive user interface using HTML, CSS, and JavaScript so that users can easily browse and play music.
- To implement backend functionality using Python and Flask framework for handling user requests, managing songs, and controlling application logic.
- To store and manage music-related data using SQLite database, including song details, artist information, and playlists.
- To provide basic music player features such as play, pause, and navigation between songs.
- To ensure smooth communication between frontend and backend for efficient retrieval and display of music data.
- To design a simple and scalable system architecture that can be enhanced in the future with additional features like user authentication, personalized playlists, and music recommendations.

SCOPE OF THE PROJECT

The scope of the Dubsvibe Music Streaming Web Application is to develop a web-based platform that allows users to access and stream music through a simple and user-friendly interface. The project mainly focuses on building a basic music streaming system that demonstrates the integration of frontend, backend, and database technologies.

The application enables users to browse available songs and play music directly from the web interface. The system is designed to provide basic music player functionalities such as play, pause, and navigation between songs. The frontend developed using HTML, CSS, and JavaScript ensures that users can interact with the application easily and enjoy a smooth listening experience.

On the server side, the application uses Python with the Flask framework to handle user requests and manage communication between the frontend and the database. The backend processes user interactions, retrieves song information, and ensures that the requested music is played correctly. This helps in maintaining proper functionality and performance of the system.

The project uses SQLite as the database to store music-related information such as song names, artist details, and file locations. The database helps in organizing and retrieving music data efficiently whenever the user selects a song to play.

The scope of this project is mainly limited to developing a basic music streaming web application for educational and demonstration purposes. However, the system can be further expanded in the future by adding advanced features such as user authentication, personalized playlists, music recommendations, search functionality, and cloud-based music storage.

In conclusion, the Dubsvibe project provides a clear understanding of full-stack web application development and demonstrates how web technologies can be used together to create an online music streaming platform.

EXISTING SYSTEM

In traditional or existing music systems, users often face several limitations when accessing and managing music. Many systems depend on downloading songs and storing them on personal devices, which consumes storage space and makes music management difficult. Users must manually organize songs, which can be time-consuming and inconvenient. Many basic music systems also lack an interactive user interface and efficient management features. Searching for songs, organizing playlists, and navigating through music collections can be difficult. In addition, traditional systems do not provide real-time interaction between users and the application, which affects the overall user experience.

DISADVANTAGES

- Requires users to download music files before listening.
- Consumes device storage space when many songs are stored.
- Limited accessibility because songs are stored only on one device.
- Manual management of music files such as organizing folders and songs.
- Poor user interface in traditional music systems.
- Difficult to search and organize songs efficiently.
- Some systems require software installation and configuration.

PROPOSED SYSTEM

The proposed system, Dubsvibe Music Streaming Web Application, is designed to overcome the limitations of traditional music systems by providing a web-based music streaming platform. Users can access and play music directly from a web browser without downloading large music files to their devices.

The system provides a user-friendly and interactive interface developed using HTML, CSS, and JavaScript, allowing users to easily browse and play songs. The application ensures smooth interaction and better usability through dynamic web features.

The backend is developed using Python with the Flask framework, which efficiently handles user requests and manages communication between the frontend and the database. This improves the performance and reliability of the system.

ADVANTAGES

- Allows users to stream music directly from the web browser.
- No need to download songs, which saves device storage.
- Provides a simple and user-friendly interface.
- Users can browse and play songs easily.
- Fast data handling using Python Flask backend.
- Efficient data storage using SQLite database.
- The system is flexible and scalable for adding new features.
- Can be accessed from different devices with internet access.

CHAPTER II

ABSTRACT

The Dekoder Music Streaming Web Application is a web-based system developed to provide users with a unique and convenient way to listen to music online. The main purpose of this project is to create a platform where users can browse and play songs through a web browser without the need to download music files.

The application is developed using modern web technologies that combine frontend, backend, and database components. The frontend of the application is built using HTML, CSS, and JavaScript, which are responsible for designing the user interface and creating interactive features. HTML is used to structure the web pages, CSS is used for styling and layout design, and JavaScript is used to implement dynamic functionalities such as music playback controls and user interactions.

The backend of the system is developed using Python with the Flask framework. Flask is a lightweight web framework that helps in handling server-side operations. It processes user requests, manages the communication between the frontend and the database, and controls the overall functionality of the application.

For data storage, the project uses SQLite, a lightweight relational database management system. SQLite is used to store important information such as song titles, artist names, album details, and the location of music files. This database helps the system retrieve and display song information quickly whenever the user interacts with the application.

Overall, the Dekoder Music Streaming Web Application demonstrates how different web technologies can be integrated to build a complete music streaming platform. The project also serves as a practical example of full-stack web development, where frontend design, backend processing, and database management work together to deliver a functional web application.

METHODOLOGY

The development of the Dekoder Music Streaming Web Application follows a structured methodology to design, develop, and implement the system effectively. The project is developed using a step-by-step process that includes requirement analysis, system design, development, testing, and deployment. This approach ensures that the application functions properly and meets the intended objectives.

The first phase of the methodology is requirement analysis. In this phase, the requirements of the system are carefully identified and analyzed. The main objective is to understand what features the application should provide to the users. The basic requirements identified for the Dekoder application include browsing songs, playing music, and viewing song details in a dashboard.

During this phase, the technologies required for the project are also selected. HTML, CSS, and JavaScript are chosen for the frontend to create an interactive user interface. Python with the Flask framework is selected for backend development due to its simplicity and flexibility. SQLite is chosen as the database because it is lightweight and easy to manage.

This phase is very important because it lays the foundation for the entire project. Proper understanding of requirements ensures that the system is developed according to user needs.

After identifying the requirements, the next phase is system design. In this phase, the overall structure of the application is planned. The system design defines how different components of the application will interact with each other.

The Dekoder application follows a client-server architecture. The frontend acts as the client, which interacts with the user, while the backend server processes the requests and communicates with the database. The database is designed to store song-related information such as title, artist, and file path.

The user interface design is also planned in this phase. The layout of the web pages, navigation structure, and placement of elements such as buttons and music controls are designed to ensure a smooth user experience.

The development phase involves the actual implementation of the system. In this phase, coding is done to build the frontend, backend, and database components.

TECHNIQUES

The development of the Dekoder Music Streaming Web Application uses several modern web development techniques to ensure smooth performance, efficient data management, and an interactive user experience. These techniques help in integrating the frontend, backend, and database components of the system.

1. FRONTEND DEVELOPMENT TECHNIQUE

The user interface of the application is developed using HTML, CSS, and JavaScript. HTML is used to create the structure of the web pages, CSS is used to design and style the interface, and JavaScript is used to add interactive features such as music playback controls, user actions, and dynamic content updates.

2. BACKEND DEVELOPMENT TECHNIQUE

The backend logic of the system is implemented using Python with the Flask framework. Flask is a lightweight web framework that helps manage server-side operations. It handles user requests, processes application logic, and communicates with the database to retrieve or store information.

3. DATABASE MANAGEMENT TECHNIQUE

The project uses SQLite as the database management system. SQLite is a lightweight relational database that stores information such as song titles, artist names, and file paths. It allows the system to retrieve data quickly and efficiently whenever required. 4. CLIENT-SERVER ARCHITECTURE

The application follows a client-server architecture, where the frontend acts as the client, and the Flask server acts as the backend. The client sends requests to the server, and the server processes those requests and sends back the appropriate response.

5. WEB-BASED MUSIC STREAMING TECHNIQUE

The system uses web-based audio playback techniques to stream music directly through the browser. This allows users to listen to songs online without downloading them to their devices.

The frontend is developed using HTML for structure, CSS for styling, and JavaScript for interactivity. Features such as displaying song lists and controlling music playback are implemented using JavaScript.

The backend is developed using Python with the Flask framework. Flask handles routing, processes user requests, and connects the frontend with the database. It ensures that data is retrieved and displayed correctly.

SQLite is used to create the database, where all song-related data is stored. The database is connected to the Flask application to enable smooth data operations.

Once the development is completed, the system undergoes testing. Testing is performed to ensure that all components of the system work correctly and efficiently.

Different types of testing are conducted, including unit testing, integration testing, and system testing. Each module is tested individually and then tested as a complete system. Errors and bugs identified during testing are fixed to improve system performance. Testing ensures that the application provides a smooth and error-free user experience.

The final phase of the methodology is implementation. In this phase, the developed system is deployed and made ready for use. The Flask server is run, and the application is accessed through a web browser. Users can interact with the system by browsing songs and playing music. The application is monitored to ensure that it functions properly in a real-time environment, completed application is deployed and made ready for use.

The system is integrated so that users can access the music streaming platform through a web browser and enjoy the available features. This methodology ensures that the Dubovibe Music Streaming Web application is developed in a systematic manner, resulting in a reliable and efficient web-based music streaming platform. After implementation, the system may require maintenance to fix any issues or improve performance. Future improvements can be made by adding new features such as user authentication, playlists, and music recommendations.

SYSTEM REQUIREMENTS

The system requirements describe the hardware and software environments needed to develop and run the Dubovibe Music Streaming Web Application. These requirements ensure that the application functions properly and provides a smooth user experience.

FUNCTIONAL REQUIREMENTS

- Users should be able to browse available songs.
- The system should allow users to play music through the web player.
- The application should retrieve song data from the database.
- The system should provide basic audio controls like Play and Pause.

NON-FUNCTIONAL REQUIREMENTS

- The system should provide a simple and user-friendly interface.
- The application should ensure fast response time when loading songs.
- The system should be reliable and easy to maintain.
- The application should be scalable for adding new features in the future.

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB
- Hard Disk: Minimum 20 GB free space
- Display: 1366 x 768 resolution monitor

- Input Devices: Keyboard and Mouse

SOFTWARE REQUIREMENTS

- Operating System: Windows 10 / Windows 11 / Linux / macOS
- Frontend Technologies: HTML, CSS, JavaScript
- Backend Technology: Python with Flask Framework
- Database: SQLite
- Web Browser: Google Chrome / Mozilla Firefox / Microsoft Edge
- Development Tools: Visual Studio Code or any code editor

MODULES

The Database Music Streaming Web Application is divided into several modules to ensure proper organization and smooth functioning of the system. Each module is responsible for handling specific tasks within the application.

1. USER INTERFACE MODULE

This module is responsible for providing the user interface of the application. It is developed using HTML, CSS, and JavaScript to create an interactive and responsive web page. Through this module, users can browse songs, view available music, and control music playback easily.

2. MUSIC LIBRARY MODULE

The music library module manages the collection of songs available in the system. It stores and organizes music files along with details such as song title, artist name, and album information. This module helps users browse and select songs from the available music list.

3. MUSIC PLAYER MODULE

This module handles the music playback functionality of the application. It allows users to play, pause, and control songs through the web-based music player. The player ensures smooth streaming of audio files directly through the browser.

4. BACKEND PROCESSING MODULE

The backend processing module is developed using Python with the Flask framework. It manages server-side operations such as handling user requests, retrieving song data from the database, and sending the required data to the frontend for display and playback.

5. DATABASE MANAGEMENT MODULE

This module is responsible for storing and managing music-related data using the SQLite database. It maintains information such as song details, artist names, and file paths. The database helps in efficient storage, retrieval, and management of music data.

These modules work together to form the complete Database Music Streaming Web Application and ensure that the system operates efficiently and provides a smooth music streaming experience.

SYSTEM DESIGN

The system design of the Database Music Streaming Web Application defines the structure and organization of the system components. It explains how the frontend, backend, and database interact with each other to provide music streaming functionality. The design focuses on creating a simple, efficient, and scalable architecture that ensures smooth communication between different parts of the application.

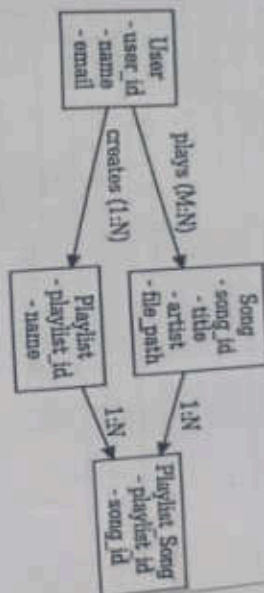
The system follows a client-server architecture. In this architecture, the client side represents the user interface that runs in the web browser, while the server side handles the application logic and database communication.

On the client side, the user interacts with the web application through the interface developed using HTML, CSS, and JavaScript. The frontend provides features such as browsing songs, selecting tracks, and controlling the music player. When a user performs an action, such as clicking the play button, a request is sent to the backend server.

The backend server is developed using Python with the Flask framework. The Flask server receives requests from the frontend, processes them, and communicates with the database to retrieve the requested information. It manages the overall workflow of the application and ensures that the requested music files and song details are sent back to the user interface.

The database layer uses SQL to store and manage music-related information. The database maintains records such as song titles, artist names, album details, and file locations. When the backend receives a request for a song, it queries the database to obtain the relevant data and sends it to the frontend for playback.

DATAFLOW DIAGRAM



SOFTWARE SPECIFICATION

The software specification describes the software tools and technologies used to develop and run the Database Music Streaming Web Application. These components help in building the user interface, managing server-side operations, and storing application data. The purpose of this specification is to ensure that all necessary software components are clearly defined so that the system can be implemented without any confusion. It also helps developers understand the tools and technologies required for building the application.

1. OPERATING SYSTEM

The application can run on different operating systems such as Windows, Linux, or macOS. These operating systems provide the environment required to develop and execute the web application.

The Database application can run on multiple operating systems. The system is designed to be platform-independent, ensuring it can work on different environments without major changes.

Supported Operating Systems:

- Windows 10 / Windows 11
- Linux (Ubuntu, Fedora, etc.)
- macOS

The application is developed using web technologies, so it can be accessed through a browser regardless of the operating system.

2. PROGRAMMING LANGUAGE

The main programming language used in the project is Python. Python is widely used for web development because it is simple, powerful, and easy to learn. It helps in implementing the backend logic of the application.

Python is chosen because:

- It is easy to learn and use
- It supports rapid development
- It has strong community support
- It integrates well with web frameworks

Python plays a major role in handling server-side logic and managing application flow.

3. WEB FRAMEWORK

The project uses the Flask Framework, which is a lightweight web framework for Python. Flask is used to handle server-side operations such as processing user requests, managing application routes, and connecting the frontend with the database.

The system uses the Flask framework for backend development.

Flask is a lightweight web framework that helps in building web applications quickly.

It provides features such as routing, request handling, and template rendering.

Advantages of Flask:

- Simple and easy to use
- Flexible and lightweight
- Suitable for small-scale applications
- Supports extensions

Flask is responsible for connecting the frontend with the database and managing user requests.

FRONTEND TECHNOLOGIES

A user interface of the application is developed using:

ML (HyperText Markup Language)

HTML is used to structure the content of the web pages. It defines elements such as headings, forms, and links.

(Cascading Style Sheets)

CSS is used to style the web pages. It controls layout, colors, fonts, and overall appearance.

script

JavaScript is used to add interactivity to the application. It handles user actions such as button clicks and music playback controls.

These technologies work together to create an interactive and user-friendly interface.

4. DATABASE MANAGEMENT SYSTEM

The application uses SQLite as the database management system. SQLite is a lightweight relational database that stores information such as song details, artist names, and file locations.

Features of SQLite:

- No server required
- Lightweight and fast
- Easy to use
- Supports SQL queries

The database stores song details such as title, artist, and file path.

5. WEB BROWSER

A modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge is required to view and test the application.

Supported Browsers:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

The browser acts as the client interface through which users interact with the application.

7. DEVELOPMENT TOOLS

A code editor such as Visual Studio Code, PyCharm, or any text editor can be used to write and manage the project source code.

Various tools are used during development:

Code Editors:

- Visual Studio Code
- PyCharm
- Sublime Text

These tools help in writing, editing, and managing source code efficiently.

Version Control (Optional):

- Git

Used to track changes and manage project versions.

SOFTWARE ARCHITECTURE TOOLS

Although not mandatory, design tools can be used for planning:

- UML diagram tools
- Flowchart tools

These tools help in visualizing system design and workflow.

LIBRARY AND PACKAGE REQUIREMENTS

The system requires certain Python libraries:

- Flask
- SQLite3 (built-in)

These libraries provide necessary functionalities for web development and database management.

INSTALLATION REQUIREMENTS

To run the system, the following steps are required:

- Install Python
- Install Flask using pip
- Create project directory

- Setup database
- Run the Flask applications

This setup ensures that the application runs correctly.

SOFTWARE CONSTRAINTS

Some limitations related to software include:

- Works best for small-scale applications
- Limited database capacity (SQLite)
- Basic security features

These constraints can be improved in future versions.

COMPATIBILITY REQUIREMENTS

The system is compatible with:

- Different operating systems
- Various web browsers
- Standard hardware configurations

This ensures wide usability of the application.

MAINTENANCE REQUIREMENTS

The software requires periodic maintenance such as:

- Updating libraries
- Fixing bugs
- Improving performance

Regular maintenance ensures smooth functioning of the system.

SECURITY REQUIREMENTS

Basic security measures include:

- Input validation
- Safe database queries
- Error handling

Future enhancements can include authentication and encryption.

PERFORMANCE REQUIREMENTS

The system should:

- Load quickly
- Respond to user actions instantly
- Handle multiple requests efficiently

Performance is important for a good user experience.

SCALABILITY REQUIREMENTS

The system can be improved for scalability by:

- Using cloud databases
- Implementing APIs
- Supporting more users

This will help in expanding the system.

RELIABILITY REQUIREMENTS

The system should function without errors and provide consistent performance. Proper testing ensures reliability.

SAMPLE SOURCE CODE

```
Backend - Python (Flask Application)

from flask import Flask, render_template, request, redirect, url_for import
sqlite3 import os
app = Flask(__name__) DATABASE =
'music.db' def get_db_connection():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row    return conn
@app.route("/") def index():
    conn = get_db_connection()    songs = conn.execute("SELECT *
FROM songs").fetchall()    conn.close()    return
render_template("index.html", songs=songs)
@app.route("/play/<int:song_id>")

def play(song_id):

    conn = get_db_connection()    song = conn.execute("SELECT * FROM songs WHERE id =
?, (song_id)).fetchone()    conn.close()    return render_template("player.html", song=song)
@app.route("/add", methods=['GET', 'POST']) def
add_song():    if request.method == 'POST':        title
= request.form['title']        artist = request.form['artist']
file_path = request.form['file']
        conn = get_db_connection()        conn.execute(
            "INSERT INTO songs (title, artist, file_path) VALUES (?, ?, ?),
            (title, artist, file_path)
        )

        conn.commit()        conn.close()        return
redirect(url_for("index"))    return
render_template("add_song.html") if __name__ ==
 '__main__':    app.run(debug=True)
```

Database Initialization (database.py)

```
import sqlite3
connection = sqlite3.connect("music.db")
cursor = connection.cursor()
cursor.execute("""
CREATE TABLE IF NOT EXISTS songs (      id INTEGER
PRIMARY KEY AUTOINCREMENT,
title TEXT NOT NULL,  artist TEXT
NOT NULL,  file_path TEXT NOT
NULL
)
""")
connection.commit() connection.close()
print("Database Created Successfully")
```

```

{{NAME}} - Home Page (index.html)
<!DOCTYPE html>
<html>
<head>
<title>Dubsvibe Music Streaming</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<h1>Dubsvibe Music Streaming</h1>
<table>
<tr>
<th>Song</th>
<th>Artist</th>
<th>Play</th>
</tr>
{% for song in songs %}
<tr>
<td>{{ song['title'] }}</td>
<td>{{ song['artist'] }}</td>
<td>
<a href="/play/{{ song['id'] }}">Play</a>
</td>
</tr>
{% endfor %}
</table>

```



```

<a href="/add">Add New Song</a>
</body>
</html>
HTML - Music Player Page (player.html)
<!DOCTYPE html>
<html>
<head>
<title>Music Player</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<h2>{{ song['title'] }}</h2>
<p>Artist : {{ song['artist'] }}</p>
<audio controls>
<source src="/static/music/{{ song['file_path'] }}" type="audio/mp3">
</audio>
<br><br>
<a href="/">Back to Home</a>
</body>
</html>
HTML - Add Song Page (add_song.html)
<!DOCTYPE html>
<html>
<head>
<title>Add Song</title>

```

```
</head>
<body>
<h2>Add New Song</h2>
<form method="POST">
<label>Song Title</label>
<input type="text" name="title" required>
<br><br>
<label>Artist Name</label>
<input type="text" name="artist" required>
<br><br>
<label>Music File Name</label>
<input type="text" name="file" required>
<br><br>
<button type="submit">Add Song</button>
</form>
</body>
</html>
```

```
CSS – Styling (style.css) body { font-  
family: Arial; background-color:  
#f5f5f5; text-align: center; } h1 { color:  
#333; }
```

```
table { margin: auto; border-collapse:  
collapse; width: 60%; } table, th, td {  
border: 1px solid black; padding:  
10px; } a { text-decoration: none;  
color: blue; }  
button { padding: 10px 20px;  
background-color: green; color:  
white; border: none; }
```

JavaScript – Basic Music Control (script.js)

```
function playSong(id)  
{ let audio = document.getElementById("audioPlayer");  
  audio.src = "/play/" + id; audio.play();  
}  
function pauseSong()  
{ let audio = document.getElementById("audioPlayer");  
  audio.pause();  
}  
function stopSong()  
{ let audio = document.getElementById("audioPlayer");  
  audio.pause(); audio.currentTime = 0;  
}
```

SCREENSHOTS

Dubvibe Music Streaming

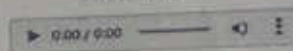
Song	Artist	Play
Song 1	Artist A	Play
Song 2	Artist B	Play
Song 3	Artist C	Play
Song 4	Artist D	Play

Add New Song

Home Page

Song 1

Artist: Artist A



Back to Home

Music Player

Add Song Page

Song Title

Artist Name

Music File Name

DB Browser for SQLite

Database Structure | [Screen Help](#) | [FAQ](#) | [Project](#) | [Donate SQL](#)

3 filters

	id	title	artist	file_path
1	1	Song 1	Artist A	song1.mp3
2	2	Song 2	Artist B	song2.mp3
3	3	Song 3	Artist C	song3.mp3
4	4	Song 4	Artist D	song4.mp3

Database Table

TESTING

During testing, different components of the system such as the frontend interface, backend processing, and database operations are checked carefully. The system is tested to ensure that users can browse songs, play music, and interact with the application without any issues.

1. UNIT TESTING

Unit testing is performed to test individual components of the system. Each module, such as the music player, song list display, and database connection, is tested separately to ensure that it works correctly.

The Duhvibe Music Streaming Web Application, unit testing is performed on different modules such as the music player, database connection, and backend functions.

Each function in the Flask backend is tested individually. For example, the function responsible for retrieving songs from the database is tested to ensure that it returns correct data. Similarly, the function for adding new songs is tested to verify that it stores data properly in the SQLite database.

Frontend components such as buttons and forms are also tested individually. For example, the play button is tested to ensure that it triggers the correct action.

Unit testing helps in identifying errors at an early stage and makes debugging easier. By testing each unit separately, the reliability of the system increases and overall performance improves.

2. INTEGRATION TESTING

Integration testing is conducted to verify the interaction between different modules of the application. It ensures that the frontend interface, backend server, and database communicate properly with each other.

Integration testing is performed to verify the interaction between different modules of the system. In the Dubsvibe application, it ensures that the frontend, backend, and database work together properly.

For example, when a user clicks on a song, the frontend sends a request to the Flask server. The backend processes the request and retrieves data from the SQLite database. This entire flow is tested to ensure smooth communication between components.

Integration testing also checks whether the data passed between modules is correct. Any mismatch or communication failure is identified and fixed during this phase.

This testing ensures that all modules are properly connected and function together as a complete system. It is an important step in ensuring the stability of the application.

3. SYSTEM TESTING

System testing checks the entire application as a complete system. It ensures that all features such as song browsing, music playback, and adding songs are functioning properly in the integrated environment.

System testing is the process of testing the complete application as a whole. It verifies that the entire Dubsvibe Music Streaming Web Application works according to the requirements.

In this phase, all functionalities such as browsing songs, playing music, and adding new songs are tested together. The system is checked to ensure that it behaves correctly under normal usage conditions.

Different scenarios are tested, such as:

- Opening the homepage
- Selecting a song
- Playing music
- Adding a new song

System testing ensures that the application meets the expected requirements and provides a smooth user experience. It also helps in identifying any issues that were not detected during unit or integration testing.

4. USER INTERFACE TESTING

User interface testing ensures that the web pages are displayed correctly and that users can easily navigate through the application. It verifies that buttons, links, and forms work as expected.

User Interface (UI) testing focuses on verifying that the application interface is userfriendly, responsive, and visually correct.

In the Dubsvibe application, UI testing ensures that all web pages are displayed properly in the browser. It checks whether elements such as buttons, links, forms, and music player controls are working correctly.

The layout, design, and navigation of the application are tested to ensure ease of use.

The system is also tested on different browsers to ensure compatibility.

UI testing is important because it directly affects user satisfaction. A well-designed and functional interface improves the overall user experience.

5. PERFORMANCE TESTING

Performance testing ensures that the application loads quickly and responds properly when users interact with the system. It checks whether the system can handle multiple user requests without slowing down.

Performance testing is conducted to evaluate how the system performs under different conditions. It checks the speed, responsiveness, and stability of the application.

In the Dubsvibe application, performance testing includes measuring the time taken to load the homepage, retrieve songs from the database, and play music.

The system is tested to ensure that it can handle multiple requests without slowing down. The database performance is also checked to ensure quick data retrieval. Performance testing helps in identifying bottlenecks and improving system efficiency. It ensures that the application provides a smooth and fast experience for users.

6. FUNCTIONAL TESTING

Functional testing is performed to verify whether the system functions according to the specified requirements. It focuses on testing the features and functionalities of the DubsVibe Music Streaming Web Application.

In this testing, each feature is checked to ensure that it produces the expected output. For example, when a user clicks on a song, the system should play the selected music correctly. Similarly, when a user adds a new song, the data should be stored properly in the database and displayed on the homepage.

Different functional test cases are executed such as:

- Checking song display
- Verifying play and pause functionality
- Validating song addition

Functional testing ensures that the system behaves correctly and meets user expectations. It is essential to confirm that all features are working as intended.

7. DATABASE TESTING

Database testing is used to verify the correctness, integrity, and performance of the database used in the system. In the DubsVibe application, SQLite database is tested thoroughly. This testing ensures that all data operations such as insertion, retrieval, and updating of records are performed correctly. For example, when a new song is added, the system checks whether the data is correctly stored in the songs table.

It also verifies:

- Data consistency
- Proper table structure
- Correct query execution

Database testing ensures that the system handles data accurately and efficiently without any loss or corruption.

8. SECURITY TESTING

Security testing is performed to ensure that the system is protected against unauthorized access and potential threats.

In the Duhsvibe application, basic security testing includes validating user inputs and preventing invalid data entry. It checks whether the system is safe from common issues such as SQL injection and improper data handling.

Although the current system is simple, security testing ensures that:

- Only valid data is accepted
- Database operations are safe
- System integrity is maintained

In future enhancements, security can be improved by adding user authentication, password protection, and encryption techniques.

FEASIBILITY STUDY

The feasibility study is an important part of the system development process. It helps to determine whether the Dubsvibe Music Streaming Web Application is practical, achievable, and beneficial to develop. It evaluates different factors such as technical resources, cost, and operational requirements to ensure that the project can be successfully implemented. The feasibility study is an essential step in the development of the Dubsvibe Music Streaming Web Application. It helps in evaluating whether the proposed system is practical, achievable, and beneficial. This study examines various aspects such as technical feasibility, economic feasibility, operational feasibility, schedule feasibility, and other supporting factors. The main purpose is to ensure that the project can be successfully implemented within the available resources and constraints.

INTRODUCTION TO FEASIBILITY STUDY

A feasibility study is conducted before the development of any system to analyze its viability. It helps in identifying potential challenges, resource requirements, and expected outcomes. In the case of the Dubsvibe application, the feasibility study ensures that the system can be developed using available tools and technologies while meeting user requirements.

The study also helps in reducing risks and improving decision-making. By analyzing different aspects of feasibility, developers can ensure that the system is efficient, cost-effective, and user-friendly.

1. TECHNICAL FEASIBILITY

Technical feasibility focuses on whether the required technology, tools, and resources are available to develop the system. Technical feasibility evaluates whether the required technology and resources are available to develop the system. The Dubsvibe application is developed using widely used technologies such as HTML, CSS, JavaScript, Python (Flask), and SQLite. These technologies are easy to learn, freely available, and suitable for developing web-based applications. Therefore, the project is technically feasible.

The frontend technologies (HTML, CSS, JavaScript) are standard web development tools that are supported by all modern browsers. The backend technology, Python with Flask, is lightweight and flexible, making it suitable for small to medium-scale web applications. SQLite is used as the database because it is simple, lightweight, and does not require a separate server. It is ideal for applications where data requirements are moderate.

The system does not require high-end hardware or complex software configurations. It can run on a basic computer with standard specifications.

Therefore, the Dubsvibe project is technically feasible as all required technologies and resources are readily available and easy to implement.

2. ECONOMIC FEASIBILITY

Economic feasibility analyzes the cost required to develop and maintain the system. The Dubsvibe project uses open-source tools and technologies, which reduces the development cost significantly. Since no expensive software or hardware is required, the project is economically feasible.

Economic feasibility evaluates the cost involved in developing and maintaining the system.

The Dubsvibe application is developed using open-source technologies, which significantly reduces development costs. Tools such as Python, Flask, and SQLite are freely available, eliminating the need for expensive licenses.

The hardware requirements are minimal, and the system can be developed using an existing computer. There is no need for additional investment in infrastructure.

Maintenance costs are also low because the system is simple and easy to manage.

Updates and improvements can be made without significant expense.

Since the project requires minimal financial investment and provides useful functionality, it is economically feasible.

3. OPERATIONAL FEASIBILITY

Operational feasibility determines whether the system will work efficiently in the real environment and whether users can easily operate it. The Dubsvibe application has a simple and user-friendly interface that allows users to browse and play music easily. This ensures that the system can be operated without difficulty.

Operational feasibility determines whether the system will function effectively in a realworld environment and whether users can operate it easily.

The Dubsvibe application is designed with a simple and user-friendly interface. Users can easily browse songs, play music, and interact with the system without requiring technical knowledge.

The system provides clear navigation and straightforward functionality, ensuring a smooth user experience. The use of a web-based platform allows users to access the application from any device with a browser.

The application meets user requirements by providing an efficient way to stream music.

Therefore, it is operationally feasible.

4. SCHEDULE FEASIBILITY

Schedule feasibility checks whether the project can be completed within the given time period. Since the Dubsvibe system is a small-scale web application using simple technologies, it can be developed and implemented within the project timeline.

The Dubsvibe Music Streaming Web Application is a small-scale project that uses simple and well-known technologies. The development process is straightforward, and each module can be implemented within a short period.

Since the project does not involve complex algorithms or large-scale infrastructure, it can be completed within the academic project timeline. Thus, the project is feasible in terms of schedule.

LEGAL FEASIBILITY

Legal feasibility ensures that the project complies with laws and regulations.

The Dubvibe application uses open-source technologies, which are legally permitted for development and usage. However, when using music files, it is important to ensure that the content is used legally and does not violate copyright laws.

For academic purposes, sample or free-to-use audio files can be used. If the system is expanded for commercial use, proper licensing must be obtained.

Therefore, the project is legally feasible with proper precautions.

SOCIAL FEASIBILITY

Social feasibility examines the acceptance of the system by users.

Music streaming applications are widely used and accepted by people. The Dubvibe system provides a similar experience in a simplified manner, making it easy for users to adopt.

The system is designed to be accessible and user-friendly, ensuring positive user acceptance.

RESOURCE FEASIBILITY

Resource feasibility checks whether the required resources such as hardware, software, and human skills are available.

The project requires basic programming knowledge and a standard computer system.

The tools used are freely available, and the skills required can be easily acquired.

Thus, the project is feasible in terms of resource availability.

RISK ANALYSIS

Every project involves certain risks. Some possible risks in this project include:

- Technical errors during development
- Data handling issues
- Limited scalability

These risks can be minimized through proper testing, debugging, and future improvements.

CONCLUSION

The feasibility study of the Dubsvibe Music Streaming Web Application shows that the project is viable and practical. It is technically possible, economically affordable, operationally efficient, and can be completed within the given time frame.

The system meets user requirements and can be developed using available resources.

Therefore, the project is feasible and suitable for implementation.

FUTURE ENHANCEMENT

The Dubvibe Music Streaming Web Application is designed as a basic music streaming platform. However, the system can be improved in the future by adding several advanced features to enhance functionality, performance, and user experience. One possible enhancement is the implementation of user authentication, where users can create accounts and log in to the system. This will allow users to maintain personal music libraries and access their favorite songs securely. Another improvement is the addition of playlist management, which will allow users to create, edit, and manage their own playlists. This feature will make it easier for users to organize songs according to their preferences.

USER AUTHENTICATION AND AUTHORIZATION

One of the most important enhancements is the implementation of user authentication and authorization. Currently, the system does not have a login or registration feature. By adding user accounts, users can create profiles, log in securely, and access personalized features.

Authentication can be implemented using secure methods such as password encryption and session management. Authorization can be used to differentiate between users and administrators. For example, administrators can add or delete songs, while regular users can only browse and play music. This enhancement will improve security and provide a personalized user experience.

PLAYLIST MANAGEMENT SYSTEM

Another important feature that can be added is playlist management. Users can be given the ability to create, edit, and delete playlists based on their preferences.

This feature will allow users to organize songs into categories such as favorites, workout songs, or relaxation music. Playlist functionality will significantly improve user engagement and satisfaction. The system can also include options to share playlists with other users.

ADVANCED SEARCH AND FILTERING

The current system provides basic browsing functionality. In the future, advanced search and filtering options can be implemented.

Users should be able to search songs by title, artist, album, or genre. Filters can be added to sort songs based on popularity, release date, or alphabetical order.

This feature will make it easier for users to find specific songs quickly, especially when the music library grows larger.

MUSIC RECOMMENDATION SYSTEM

A recommendation system can be implemented to suggest songs to users based on their listening history.

This can be achieved using basic algorithms or machine learning techniques. The system can analyze user behavior and recommend songs that match their interests.

This feature will enhance user experience and increase engagement with the application.

ONLINE MUSIC UPLOAD FEATURE

Currently, songs are added manually through the backend. In the future, an online upload feature can be added where users or administrators can upload music files directly through the web interface.

This feature will make the system more dynamic and allow easy expansion of the music library.

Proper validation should be implemented to ensure that only supported file formats are uploaded.

MOBILE RESPONSIVENESS

Although the application can run on a browser, it can be further improved to support mobile devices more effectively.

By implementing responsive design techniques, the system can adapt to different screen sizes, such as smartphones and tablets. This will provide a better user experience across all devices.

CLOUD INTEGRATION

The current system uses local storage for music files and database management. In the future, cloud services can be integrated.

Cloud storage can be used to store music files, and cloud databases can be used to manage large amounts of data. This will improve scalability and accessibility.

Users will be able to access the system from anywhere with an internet connection.

HIGH-QUALITY AUDIO STREAMING

The system can be enhanced to support high-quality audio streaming. Different audio quality options such as low, medium, and high can be provided.

This will allow users to choose the quality based on their internet speed and preferences.

REAL-TIME STREAMING AND BUFFERING OPTIMIZATION

Future improvements can include advanced streaming techniques such as buffering and adaptive streaming.

These techniques will ensure smooth playback even when the internet connection is slow or unstable.

INTEGRATION WITH EXTERNAL APIS

The application can be integrated with external music APIs to access a large collection of songs.

APIs can provide features such as song metadata, album covers, and artist information.

This will enhance the functionality and richness of the application.

USER INTERFACE IMPROVEMENTS

The current user interface is simple. In the future, it can be improved with *modern design* elements such as animations, transitions, and better layouts.

A visually appealing interface will attract more users and improve usability.

OFFLINE MODE

An offline mode can be implemented to allow users to download songs and listen to them without an internet connection.

This feature will increase the usability of the application.

MULTI-LANGUAGE SUPPORT

The system can be enhanced to support multiple languages. This will make the application accessible to a wider audience.

Users can select their preferred language from the settings.

SOCIAL SHARING FEATURES

Users can be allowed to share songs or playlists on social media platforms. This feature will increase user engagement and help promote the application.

ADMIN DASHBOARD

An admin dashboard can be developed to manage the system efficiently.

The dashboard can include features such as:

- Adding and removing songs
- Viewing user activity
- Managing playlists

SECURITY ENHANCEMENTS

Future versions of the system can include advanced security features such as:

- Data encryption
- Secure login systems
- Protection against cyber attacks

PERFORMANCE OPTIMIZATION

The system can be optimized to handle large amounts of data and multiple users. Techniques such as caching, database indexing, and load balancing can be implemented.

CONCLUSION

The DubsVibe Music Streaming Web Application is developed to provide a simple and efficient platform for streaming music through a web browser. The main objective of this project is to design and implement a web-based system where users can easily browse and play songs without downloading them. The system demonstrates how modern web technologies can be integrated to build a functional and user-friendly music streaming application. The application is developed using HTML, CSS, and JavaScript for the frontend, which provides an interactive and responsive user interface. The backend is implemented using Python with the Flask framework, which manages server-side operations and handles communication between the frontend and the database. The SQLite database is used to store and manage music-related data such as song titles, artist names, and file paths. The project successfully demonstrates the working of a basic music streaming platform with features such as song browsing, music playback, and simple data management. It also helps in understanding the concept of full-stack web development, where frontend, backend, and database technologies work together to form a complete web application. Overall, the DubsVibe Music Streaming Web Application meets its objectives by providing a simple, reliable, and efficient system for online music streaming. The project also provides a strong foundation for future improvements and the addition of advanced features to enhance the system further.

REFERENCES

- Python Official Documentation – <https://www.python.org/doc/>
- JavaScript Tutorial – <https://www.w3schools.com/js/>
- VisualStudioCodeDocumentation – <https://code.visualstudio.com/docs>
- Web Development with Python and Flask – Technical learning resource for Flask web applications.
- Grinberg, M. *Flask Web Development*.
- Ashley, D. *Dynamic Web Pages with Python*.
- *HTML and CSS: Design and Build Websites* – by Jon Duckett